

 AULA 10 · COLABORAÇÃO

Criando e trocando branches

A partir de agora você começa a trabalhar como equipes profissionais: criando áreas de trabalho isoladas para cada funcionalidade e alternando entre elas com segurança.

GIT BRANCH

Em qual branch estou?

 bash — branch atual

```
$ git branch
```

```
* main
```

```
# o asterisco (*) indica a branch atual
```



`git branch` mostra todas as branches e **destaca a atual**.

 CRIAR BRANCH

Criar não é o mesmo que trocar

 bash — criar uma branch

```
$ git branch feature-contato
$ git branch
* main
feature-contato
# ela foi criada, mas ainda estamos na main
```



`git branch` cria uma branch, mas não muda para ela.

Trocando de branch

● ● ● bash — mudar de branch

```
$ git switch feature-contato  
Switched to branch 'feature-contato'
```

● ● ● atalho: criar + trocar

```
$ git switch -c feature-sobre  
# -c é abreviação de --create (cria a branch E já muda para ela)
```

💡 `git switch -` volta rapidinho para a última branch onde você estava.

💡 Em tutoriais antigos você verá `git checkout` — hoje prefira `git switch`.

 CADA BRANCH, SUA EVOLUÇÃO

A alteração "some" e "volta"

 demonstração

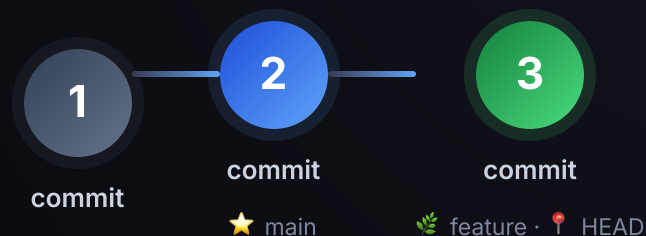
```
# na feature-contato: adiciona a seção Contato e faz commit
$ git switch main
# abra o README → a seção Contato desapareceu!
$ git switch feature-contato
# abra o README → a seção Contato reapareceu!
```



Cada branch tem seu próprio histórico e suas próprias alterações.

 **BRANCH = PONTEIRO**

O que é uma branch, por dentro



Uma branch é só um **marcador que aponta para um commit**. Ao trocar de branch, o `HEAD` (Aula 5) se move junto — por isso os arquivos mudam.

 MERGE LOCAL

Trazendo a branch de volta para a main

```
bash — juntar no seu computador

# terminei o contato.md na branch feature-contato
$ git switch main
$ git merge feature-contato
Fast-forward · contato.md atualizado na main
```



Você não precisa do GitHub para integrar: `git merge` junta a branch na main ali mesmo, no seu repositório.

✦ BOAS PRÁTICAS

Organizando suas branches



Nomes descritivos

`feature-login` em vez de `teste` ou `branch1`.

1

Uma branch por funcionalidade

Evite misturar várias coisas na mesma branch.



Verifique onde você está

Antes de trabalhar, rode `git branch`.

 PRÁTICA GUIADA

Faça você mesma

- ✓ Criar `feature-sobre` e trocar para ela.
- ✓ Adicionar uma seção ao README e fazer um commit.
- ✓ Voltar para a `main` e ver a alteração sumir.
- ✓ Voltar para `feature-sobre` e ver a alteração voltar.

ENCERRAMENTO

Branches isolam o trabalho

Você aprendeu a criar branches e alternar entre elas — desenvolvendo novas funcionalidades sem alterar a versão principal.



Você sabia?

Em muitas empresas, ninguém faz commits diretamente na `main`. Cada funcionalidade começa em uma branch própria, o que facilita a revisão e permite que várias pessoas trabalhem ao mesmo tempo.



Próxima aula: o fluxo de trabalho com **Feature Branch**.